OptiCrop: Smart Agricultural Production Optimization Engine

Project Description:

OptiCrop harnesses Machine Learning to give farmers and agricultural advisors a fast, data-driven crop recommendation. The platform includes a Prediction Engine module, which recommends the single best-suited crop from seven soil and climate parameters, and an About module that explains the dataset and methodology behind each recommendation.

Utilizing a Scikit-learn Logistic Regression model trained on the Crop_recommendation dataset, OptiCrop processes Nitrogen, Phosphorus, Potassium, temperature, humidity, pH, and rainfall values to deliver an instant, accurate crop recommendation. Built with Flask and rendered through Jinja2 templates, the platform ensures a lightweight, easy-to-use experience that helps reduce guesswork in crop selection and improve agricultural yield.

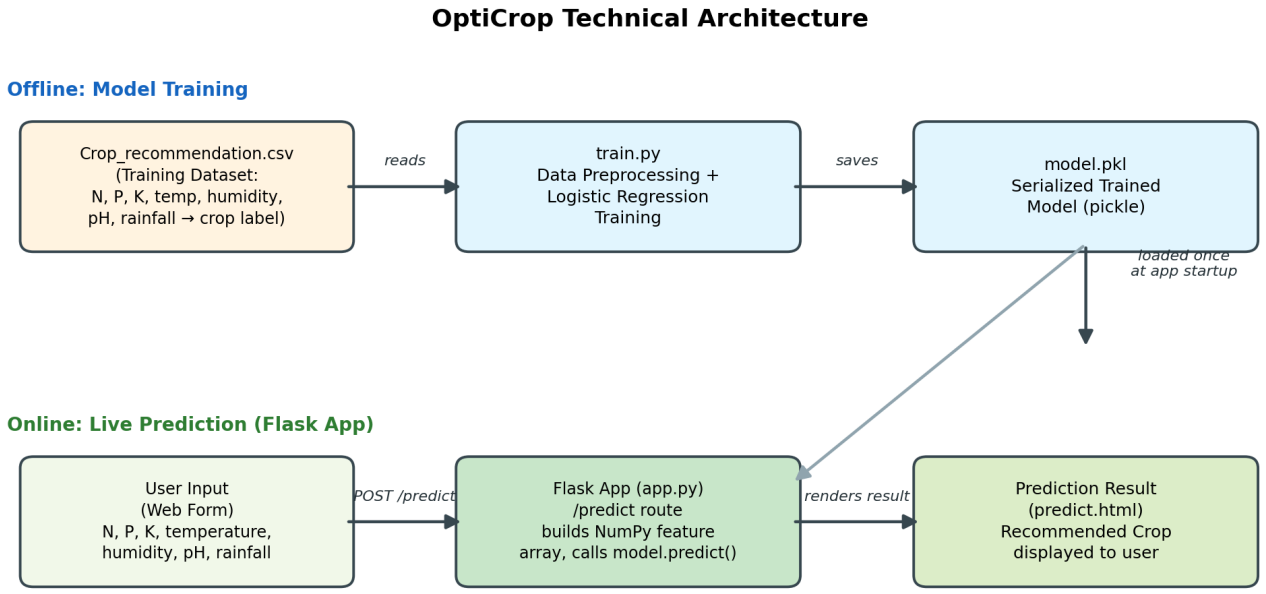
Scenarios

Scenario 1: A farmer in a humid coastal region enters high nitrogen, high rainfall, and a near-neutral pH reading. OptiCrop's model recognizes the pattern associated with water-intensive crops and recommends rice as the optimal choice.

Scenario 2: A farmer with sandy, low-rainfall soil enters low nitrogen, high temperature, and low humidity values. The system recommends a drought-tolerant crop such as mothbeans, suited to arid growing conditions.

Scenario 3: An agricultural extension officer tests several parameter combinations for a hilly district to compare recommendations across different pH ranges, using the results to advise local farmers on the most suitable crops for varying plots of land.

Technical Architecture:



Description: OptiCrop uses a compact two-tier architecture to deliver rapid, ML-driven crop recommendations. The Flask backend (app.py) loads the pre-trained model (model.pkl) once at startup and exposes it through the /predict route, which accepts form input, formats it into a 7-feature array, and returns a rendered result page. The model itself is produced separately by the training pipeline (train.py), which reads and preprocesses Crop_recommendation.csv before fitting a Logistic Regression classifier.

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Scikit-learn Familiarity:** Scikit-learn Documentation
- **Pandas / NumPy Proficiency:** Pandas Documentation, NumPy Documentation
- **HTML, CSS Skills:** W3Schools HTML/CSS Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide

Project Workflow:

Milestone 1: Data Collection and Preprocessing

- **Activity 1.1:** Source the Crop_recommendation dataset containing N, P, K, temperature, humidity, pH, and rainfall readings across 22 crop classes.
- **Activity 1.2:** Explore the dataset for class balance, missing values, and feature ranges using Pandas.
- **Activity 1.3:** Split the dataset into training and testing sets to evaluate model generalization.

Milestone 2: Model Training

- **Activity 2.1:** Select Logistic Regression as the classification algorithm for its speed and interpretability on tabular agricultural data.
- **Activity 2.2:** Train the model in train.py, evaluate its accuracy on the held-out test set, and serialize the fitted model to model.pkl using pickle.

Milestone 3: Flask Backend Development (app.py)

- **Activity 3.1:** Define the core routes - / (home), /about, and /predict (GET/POST) - and load model.pkl once at application startup.
- **Activity 3.2:** Implement the /predict handler to parse form fields (N, P, K, temperature, humidity, ph, rainfall), format them into a NumPy feature array, and run model.predict().
- **Activity 3.3:** Add error handling so the app degrades gracefully if the model file is missing or an input cannot be parsed.

Milestone 4: Frontend Development

- **Activity 4.1:** Build the shared layout.html base template and extend it in index.html, predict.html, and about.html using Jinja2.
- **Activity 4.2:** Style the interface with static/css/style.css for a clean, farmer-friendly form layout.
- **Activity 4.3:** Render the prediction result dynamically in predict.html via the prediction_text template variable.

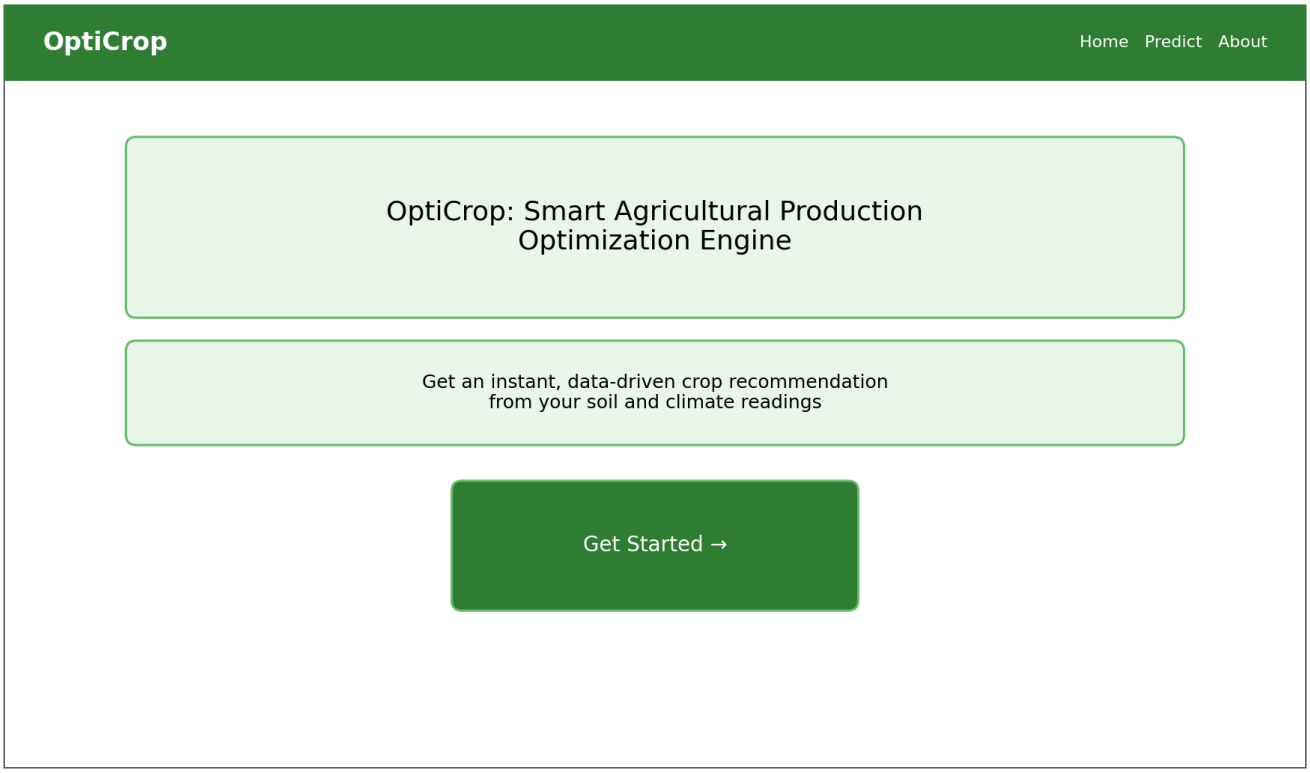
Milestone 5: Testing and Deployment

- **Activity 5.1:** Test the /predict route locally with varied soil and climate inputs to confirm correct crop recommendations.
- **Activity 5.2:** Run the application locally via python app.py on port 5000, with a future roadmap toward Gunicorn-based production deployment.

Milestone 6: Conclusion

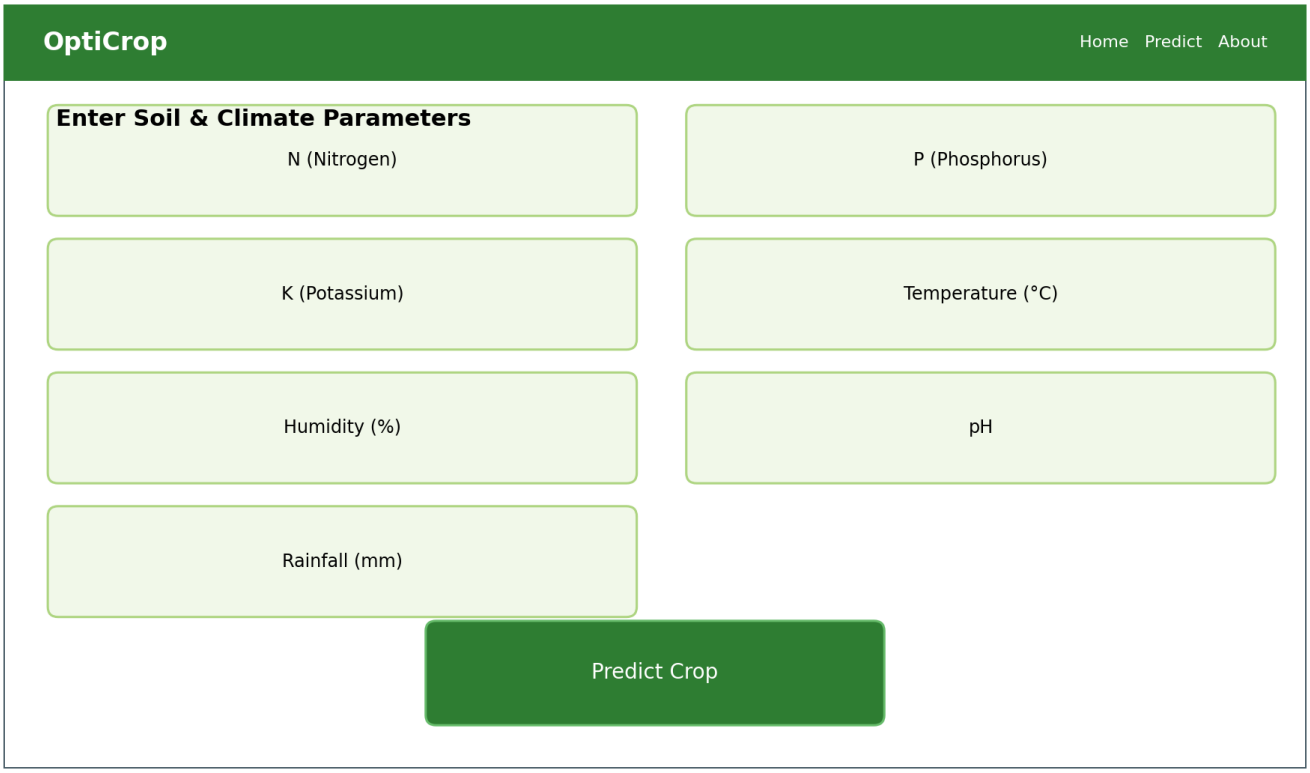
Exploring the Website's Web Pages:

Home Page:



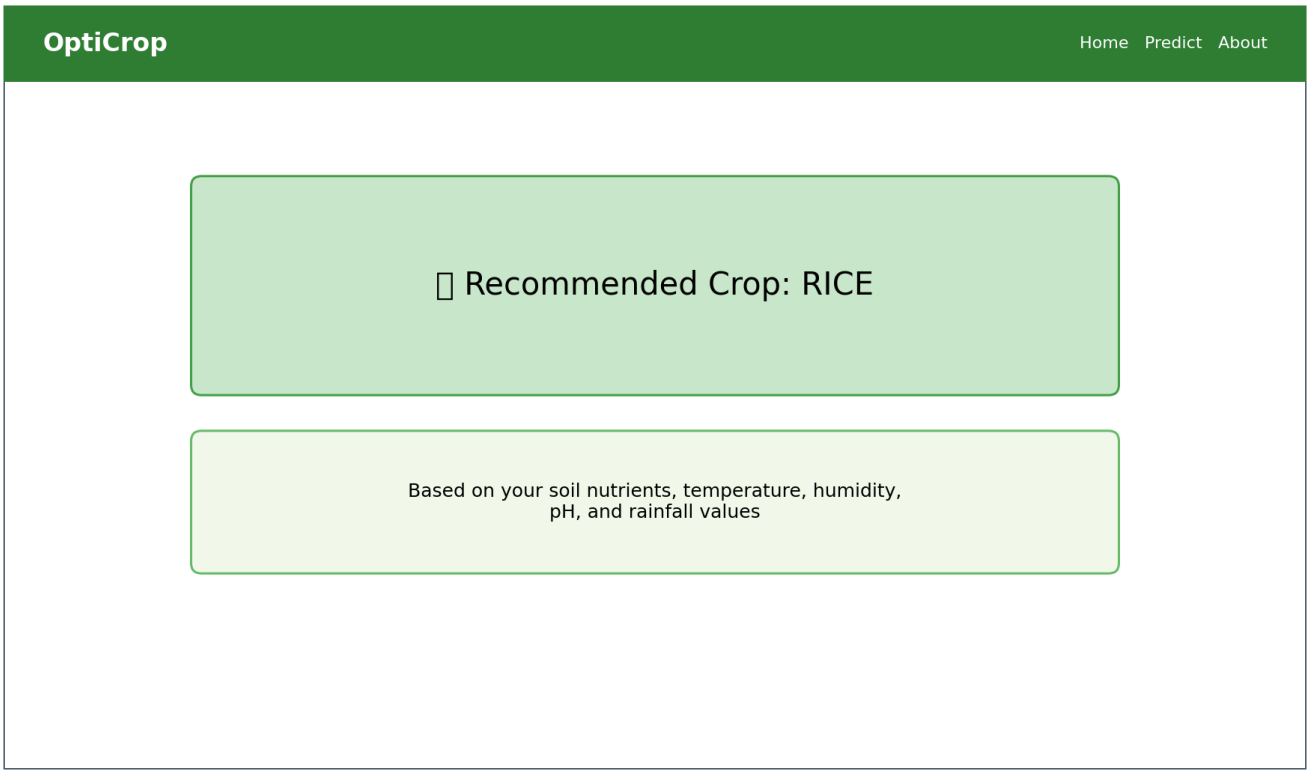
Description: The landing page introduces OptiCrop and directs users to the prediction form. It is served by the home() route and rendered via index.html.

Prediction Form Page:



Description: Users enter the seven soil and climate parameters here. On submission, the form POSTs to /predict, which runs the model and re-renders this same page with the recommendation shown above the form.

Prediction Result:



Description: The recommended crop is displayed prominently at the top of the predict.html page immediately after form submission, giving the farmer a clear, actionable result.

Conclusion:

OptiCrop is a machine learning platform built with Flask that streamlines crop selection for farmers and agricultural advisors. It uses a Scikit-learn Logistic Regression model trained on soil and climate data to instantly recommend the most suitable crop from seven input parameters. The project, which included data preprocessing, model training, and a lightweight Flask deployment, highlights how targeted ML models can support real-world agricultural decision-making with minimal infrastructure. Looking ahead, OptiCrop offers significant opportunities for expansion, such as integrating live weather and IoT sensor data, supporting multi-crop yield forecasting, and moving to a scalable cloud deployment. By continuing to refine the model and broaden its data sources, the platform is well-positioned to evolve from a recommendation tool into a comprehensive agricultural decision-support system.